



사용자 활동 시간대 분석

📅 작성일	2026년 7월 23일
📁 과목명	해커의 프로그래밍
⚙️ 과제	완료
≡ 이름	4기 39반 박성현

Q1. 모든 글, 모든 댓글에서 사용자별 활동시간을 추측해 KST 시간대로 차트로 표현하기

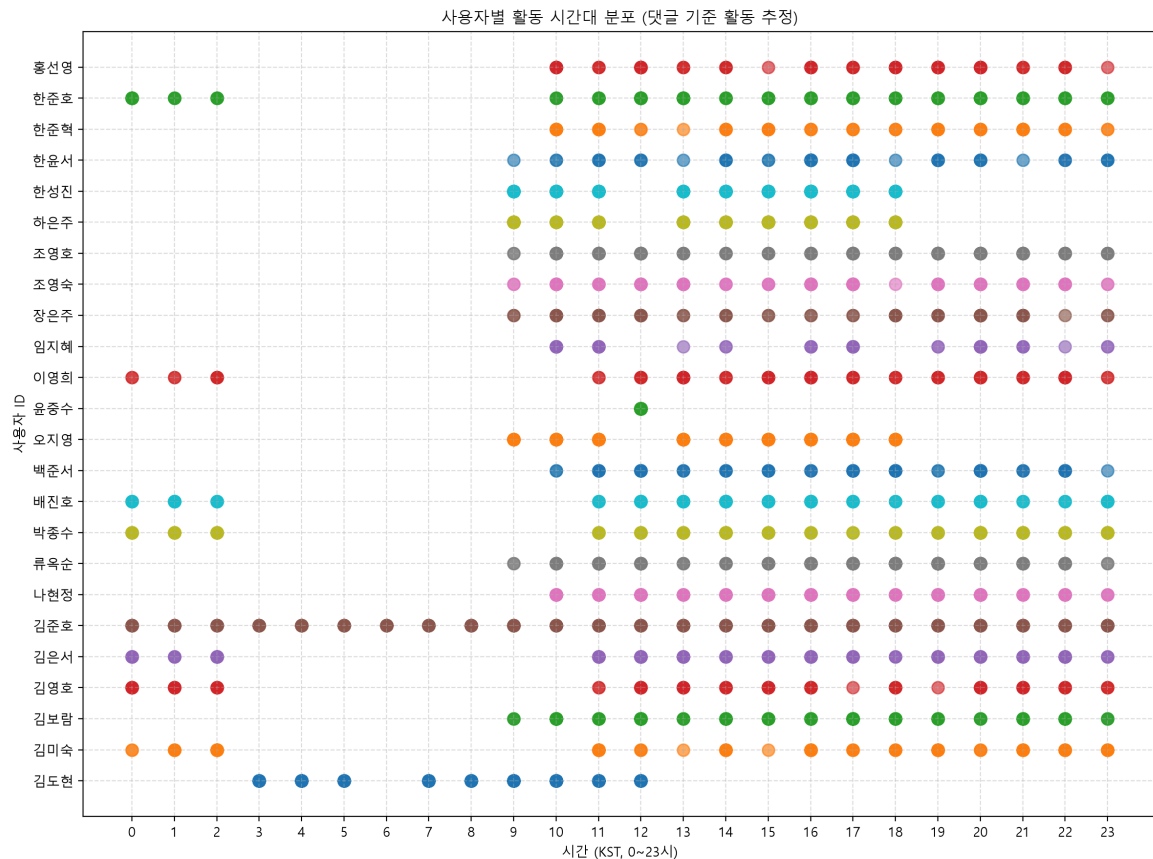
게시글을 확인해 보니 작성 시간은 바로 확인할 수 없었고, 댓글에는 `created_at` 값이 포함되어 있었다. 그래서 댓글 작성 시간을 이용해 사용자가 주로 언제 활동하는지를 추정하는 방식으로 분석을 진행했다.

분석 과정은 다음과 같다.

1. 게시판 목록을 순회하면서 게시글 ID를 수집한다.
2. 각 게시글의 댓글 API를 호출하여 댓글 목록을 가져온다.
3. 댓글에서 `author_name` 과 `created_at` 값을 추출한다.
4. `created_at` 을 KST 기준 시간으로 변환한 뒤 시간(hour)만 따로 저장한다.
5. 사용자별 활동 시간을 산점도로 시각화하여 어떤 시간대에 활동이 많은지 확인한다.

이번 과제에서 사용한 데이터는 서버 로그가 아니라 댓글 작성 시간이다. 따라서 사용자가 실제로 로그인한 시간이나 사이트에 접속한 시간을 알 수 있는 것은 아니며, 댓글을 작성한 시간을 기준으로 활동 시간대를 추정했다고 보는 것이 맞다.

결과 차트



Q2. SNS 커뮤니티라고 가정했을 때, 비정상적 활동패턴을 보이는 사용자를 모두 식별하고 그 이유 서술하기

사용자들의 활동 시간을 비교해 보니 일부 계정은 특정 시간대에 활동이 몰려 있는 모습을 확인할 수 있었다.

코드에서는 다음 두 가지 조건을 기준으로 이상 활동 후보를 찾았다.

- 활동의 절반 이상이 같은 시간대에 몰려 있는 경우
- 가장 많이 활동한 시간이 새벽(00~05시)인 경우

분석 결과 이상 활동 후보로 분류된 사용자는 다음과 같다.

- 김도현
- 김미숙
- 김준호
- 윤중수

이 사용자들은 특정 시간대에 댓글을 집중적으로 작성하거나 새벽 시간대 활동이 많았다. 물론 이것만으로 봇이나 악성 사용자라고 단정할 수는 없다. 단순히 생활 패턴이 다를 수도 있기 때문에, 실제 서비스라면 로그인 기록이나 IP 같은 다른 정보도 함께 확인해야 조금 더 정확한 판단이 가능할 것이다.

Q3. 직접 드러나지 않은 데이터를 이용한 현실 사례

서비스에서는 원래 제공하려던 기능과는 별개로, 사용하면서 남는 데이터를 분석해 다른 정보를 얻는 경우가 있다.

예를 들어 댓글은 사용자들이 의견을 남기기 위한 기능이지만, 댓글 작성 시간을 모으면 다음과 같은 정보도 확인할 수 있다.

- 주로 활동하는 시간대
- 비정상적인 활동 패턴
- 자동화 계정 여부
- 사용자 이용 습관

대표적인 사례로 **Strava Heat Map 사건**이 있다.

Strava는 운동 기록을 지도 형태로 보여주는 서비스인데, 공개된 운동 기록을 분석한 결과 군사 기지 내부 이동 경로나 시설 위치가 드러난 일이 있었다. 원래는 운동 기록을 보여주기 위한 기능이었지만, 여러 데이터를 모아서 분석하니 예상하지 못한 정보까지 확인할 수 있었던 사례이다.

이번 과제도 비슷하게 댓글 작성 시간을 이용해 사용자의 활동 패턴을 추정했다는 점에서 같은 유형의 분석이라고 생각한다.

Q4. board.nyan101.com의 구현 방식 추정

사이트 설명을 보면 오랫동안 운영하면서도 서버 비용이 거의 들지 않는다고 되어 있다.

직접 사용해 본 느낌으로는 게시글과 댓글을 단순한 API 형태로 제공하고 있고, 로그인이나 복잡한 기능도 많지 않아서 전체적으로 가벼운 구조인 것 같다.

또한 서버리스 환경이나 정적 페이지와 간단한 백엔드를 함께 사용하는 방식이라면 운영 비용을 많이 줄일 수 있을 것으로 생각된다.

서비스 구조가 단순하기 때문에 관리해야 할 부분도 적고, 서버 비용도 낮게 유지할 수 있었을 것으로 보인다.

다만 운영 비용이 적다고 해서 보안 문제가 없는 것은 아니다. 입력값 검증이 부족하다면 XSS나 SQL Injection 같은 취약점이 발생할 수도 있고, 과도한 크롤링이나 자동화 요청에 대한 대비도 필요할 것으로 생각된다.

코드

```
import re
import time

import requests
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'Malgun Gothic'
plt.rcParams['axes.unicode_minus'] = False

BASE_URL = 'http://board.nyan101.com'

def get_activity_data(max_pages=20, delay=0.3):
    """
    실제 사이트 구조에 맞춰 목록 페이지에서 post id를 수집하고,
    각 게시글의 /comments/{post_id} JSON을 읽어 사용자별 활동시간을 추정
    한다.
    """
    session = requests.Session()
    activity_rows = []

    for page_num in range(1, max_pages + 1):
        list_url = f'{BASE_URL}/list/{page_num}'
        try:
            page = session.get(list_url, timeout=20)
            page.raise_for_status()
        except Exception as e:
            print(f'페이지 수집 실패: {list_url} ({e})')
            break

        post_ids = re.findall(r'/post/(\d+)', page.text)
        if not post_ids:
            print(f'페이지 {page_num}에서 더 이상 게시글을 찾지 못했습니
다.')
            break

        unique_post_ids = list(dict.fromkeys(post_ids))
        print(f'페이지 {page_num}: 게시글 {len(unique_post_ids)}개
```

수집')

```
for post_id in unique_post_ids:
    comments_url = f'{BASE_URL}/comments/{post_id}'
    try:
        comments_res = session.get(comments_url, timeout=2
0)

        comments_res.raise_for_status()
        comments = comments_res.json()
    except Exception as e:
        print(f'댓글 수집 실패: {comments_url} ({e})')
        continue

    for item in comments:
        created_at = item.get('created_at')
        author_name = item.get('author_name')
        if created_at and author_name:
            activity_rows.append({
                'user': author_name,
                'time_str': created_at,
                'type': 'comment'
            })

    time.sleep(delay)

time.sleep(delay)

return activity_rows

def process_and_visualize(activity_rows):
    if not activity_rows:
        print('수집된 데이터가 없습니다. 사이트 응답 또는 경로를 다시 확인
하세요.')
        return

    df = pd.DataFrame(activity_rows)
    df['timestamp'] = pd.to_datetime(df['time_str'], format='%Y-%m
-%d %H:%M', errors='coerce')
    df = df.dropna(subset=['timestamp']).copy()

    # 공개된 생성시각이 없으므로, 댓글 생성시각을 활동 시간의 대표 지표로 사
용
```

```

df['timestamp_kst'] = df['timestamp'].dt.tz_localize('Asia/Seoul')
df['hour'] = df['timestamp_kst'].dt.hour

user_hour_counts = (
    df.groupby(['user', 'hour']).size().reset_index(name='count')
)
user_hour_counts = user_hour_counts.sort_values(['user', 'hour'])

plt.figure(figsize=(12, 9))

user_ids = sorted(df['user'].unique())
user_to_y = {user: i for i, user in enumerate(user_ids)}

suspicious_users = []
for user in user_ids:
    user_df = df[df['user'] == user]
    hour_counts = user_df['hour'].value_counts().sort_index()
    dominant_hour = hour_counts.idxmax()
    dominant_ratio = hour_counts.max() / len(user_df)

    # 비정상적 패턴: 특정 시간대에 과도하게 몰리거나, 심야에만 집중하는
    경우
    if dominant_ratio >= 0.5 or dominant_hour in [0, 1, 2, 3, 4, 5]:
        suspicious_users.append(user)

    plt.scatter(user_df['hour'], [user_to_y[user]] * len(user_df), s=80, alpha=0.65)

plt.yticks(range(len(user_ids)), user_ids)
plt.xlabel('시간 (KST, 0~23시)')
plt.ylabel('사용자 ID')
plt.title('사용자별 활동 시간대 분포 (댓글 기준 활동 추정)')
plt.xticks(range(0, 24))
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('user_activity_chart.png', dpi=200)

print('차트가 user_activity_chart.png 로 저장되었습니다.')
if suspicious_users:

```

```
print('잠재적 이상 사용자 후보:', ', '.join(sorted(set(suspicious_users))))
```

```
if __name__ == '__main__':  
    raw_data = get_activity_data(max_pages=20, delay=0.3)  
    process_and_visualize(raw_data)
```